Survey of Vector Database Management Systems

James Pan[1], Jianguo Wang[2], Guoliang Li[1]

[1]Department of Computer Science, Tsinghua University

[2]Department of Computer Science, Purdue University

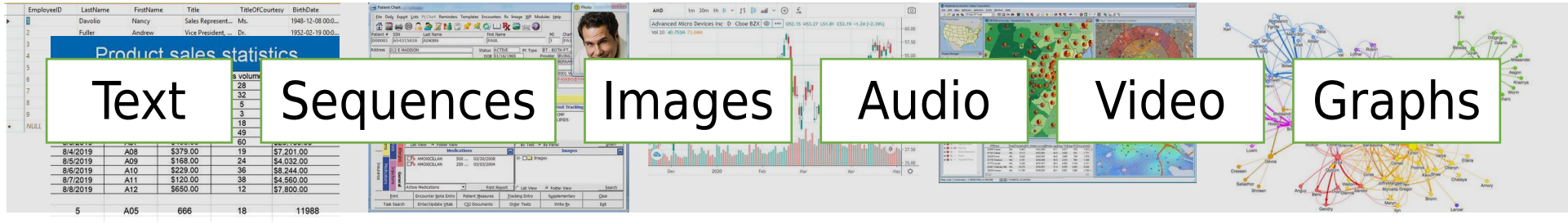
Embeddings: Building Blocks of Future

More and more applications rely on embedding vectors that are retrieved via similarity search

- Forecasting
- Classification
- RecSys
- Pattern Recognition
- Reinforcement Learning
- Object Detection
- Semantic Retrieval
- Segmentation
- Generation

DeepMind | Hugging Face | OpenAI

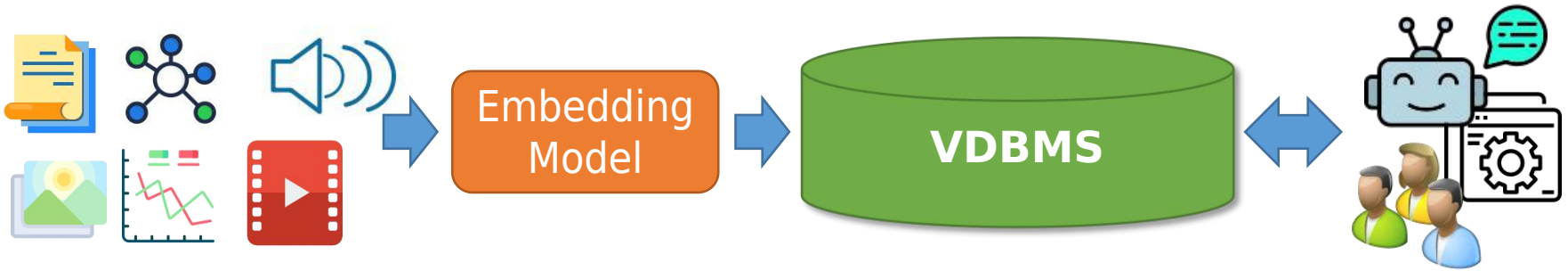
- MiniLM-L6
- MPNet-Base
- ResNet-50
- CLIP VIT
- Whisper
- Multimodal Receiver



Vector Databases

More and more applications rely on embedding vectors that are retrieved via similarity search

Goal: Store embeddings in a DB and retrieve for whatever downstream task



Example:
Vector DB + LLMs
- RAG
- Semantic Cache
- Auto Few-Shot Examples

Search Capabilities	Performance	Quality	System-Level Features
• Support various queries, e.g. k-NN, Range NN, Predicated (Filtered) Search	• Low latency queries • High throughput • Memory efficient	• High recall • High semantic relevance	• Scalable • Available • Elastic • Consistent • Persistent

Vector Databases: Key Challenges

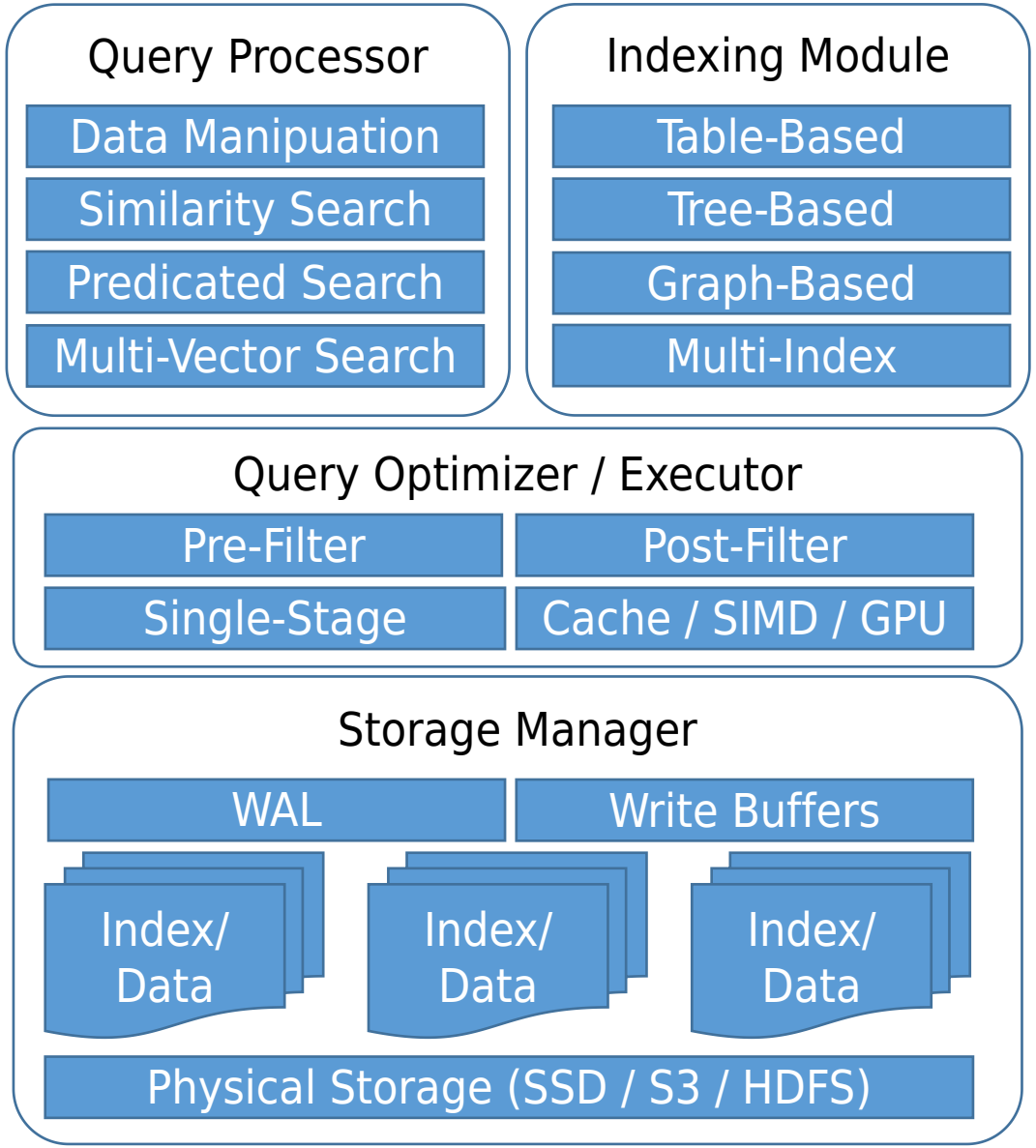
More and more applications rely on embedding vectors that are retrieved via similarity search

Goal: Store embeddings in a DB and retrieve for whatever downstream task

1) ***<u>Size</u>: Existing hardware (e.g. cache lines) not for large data units***
2) ***<u>Structure</u>: Lack of structure (e.g. attrs) makes embeddings hard to index***
3) ***<u>Semantics</u>: Mathematical similarity ≠ semantic similarity***

Search Capabilities	Performance	Quality	System-Level Features
• Index design to support various search queries	• Cache thrashing • More frequent paging • Expensive similarity operator	• Low-Recall ANNS • Low Relevance similar to 	• Partitioning/replication for scalability and load balancing • Physical storage for persistence but also minimizes I/O during queries • Maintaining consistent reads during slow updates

Vector Databases: Architecture



Component	Techniques
Query Processor	• Various similarity scores (relevance) • Exact & Approximate Algorithms for various query types (k-NN, Range NN, Predicated Queries, Multi-Modal, etc.) • Distributed Search Algorithms
Indexing	• Table (Hash) Based: k-Means (SPANN), IVFPQ, LSH • Tree Based: k-d Tree, Principal Component Trees, Random Projection Trees (ANNOY) • Graph Based: HNSW, Vamana, NSG
Query Optimizer / Executor	• Predicated Query Planning: Pre-filtering, Post-filtering, Single-stage filtering • Cost-Based Plan Selection, Rule-Based Plan Selection • Hardware Acceleration: SIMD Shuffle, GPU • Distributed / Disaggregated Execution
Storage Manager	• Disk-Based Indexes • Storage schemes for R/W Workloads: LSM Tree, Segments / Slices • Distributed Storage: Shards / Replicas • Quantization

Existing Vector Databases

Build a capable system for high-performance & high-quality search with system-level features

	Examples	Key Features	Key Design Aims
Search Engines & Libraries	• Apache Lucene • Meta FAISS	• Aimed at **specific search capability** • No system-level features	• Optimize a given search capability by adopting techniques for **high-performance** and **high-quality search**
Native Systems	• Pinecone • Milvus/Zilliz • Weaviate	• May offer **multiple search capabilities** • **System-level features**, e.g. scalable, available, federated, persistent, consistent	• Use techniques that allow **system-level features**, e.g. distributed storage and indexes, replication, batched updates • Use **optimizations techniques** to support predicated queries
Extended Systems	• Redis • PostgreSQL • GaussDB	• **Constrained by need to support other queries**, e.g. relational queries	• Use techniques that can **adapt to existing database structure**, e.g. storage structure

Search Engines & Libraries

Provide high-performance & high-quality search for a specific search capability

	Query Processor	Storage & Indexing	Query Optimizer	Query Executor
Performance	• Fundamentally efficient index-based algorithms	• Storage schemes for disk-based indexes • Quantization	N/A	• Hardware acceleration
Quality	• Exact algorithms	• High-quality indexes	N/A	N/A

	Query Processor	Storage & Indexing	Query Optimizer	Query Executor
Apache Lucene	• HNSW k-NN	• Disk-resident HNSW based on segments	N/A	• GPU Acceleration (Issue #13003)
Meta FAISS	• Naive k-NN • IVF, IVFPQ k-NN • LSH k-NN • HNSW k-NN	• In-memory indexes • Memory-mapped IVF • Product quantization	N/A	• GPU Acceleration • Scatter-gather

Native & Extended Systems

- ***Native: Provide system-level features and support wide range of search capabilities***
- ***Extended: Integrate vector search capabilities within existing data management system***

	Query Processor	Storage & Indexing	Query Optimizer	Query Executor
Performance	• Fundamentally efficient index-based algorithms	• Storage schemes for disk-based indexes • Quantization	N/A	• Hardware acceleration • **R/W Disagg.** • **CPU/Store Disagg.**
Quality	• Exact algorithms	• High-quality indexes	N/A	N/A
Capabilites	• **Multiple search algorithms**	• **Multiple index choices**	• **Predicated query plan**	N/A
Scalability (Partitioning)	• **Distributed search**	• **Distributed indexes** • **Distributed storage**	N/A	• **Distributed execution**
Availability	N/A	• **Replication**	N/A	N/A
Consistency	N/A	N/A	N/A	• **Concurrency Ctrl.** • **Batched updates**

Native Systems

Provide system-level features to support fast and accurate wide range of search capabilities

	Query Processor	Storage & Indexing	Query Optimizer	Query Executor
Pinecone Pinecone	• k-NN • Predicated k-NN • Keyword Search • Reranking	• Pods, Shards, & Shard Replicas • Tiered slabs • On-disk blob storage (backup & persistence) • Proprietary indexes	N/A	• Scatter-gather • R/W Disagg. • Log-backed durable writes • Batched updates
Milvuz/Zilliz milvus	• k-NN • Range NN • Predicated k-NN • Multi-vector search • Grouping Search • Full Text Search • Reranking	• Replicas • Data Partitions • Multiple index types • Blob storage (backup & persistence)	• Cost-based predicated query planning	• Scatter-gather • R/W Disagg. • CPU/Store Disagg. • Log-backed durable writes • Cache & SIMD acceleration • GPU acceleration • Tunable consistency
Weaviate Weaviate	• k-NN • Predicated k-NN • Keyword Search • Reranking	• Document store • Shards, Replicas, Snapshots (backup) • LSM Tree • HNSW, ACORN • Quantization	N/A	• Scatter-gather • Log-backed durable writes • Quorum consistency

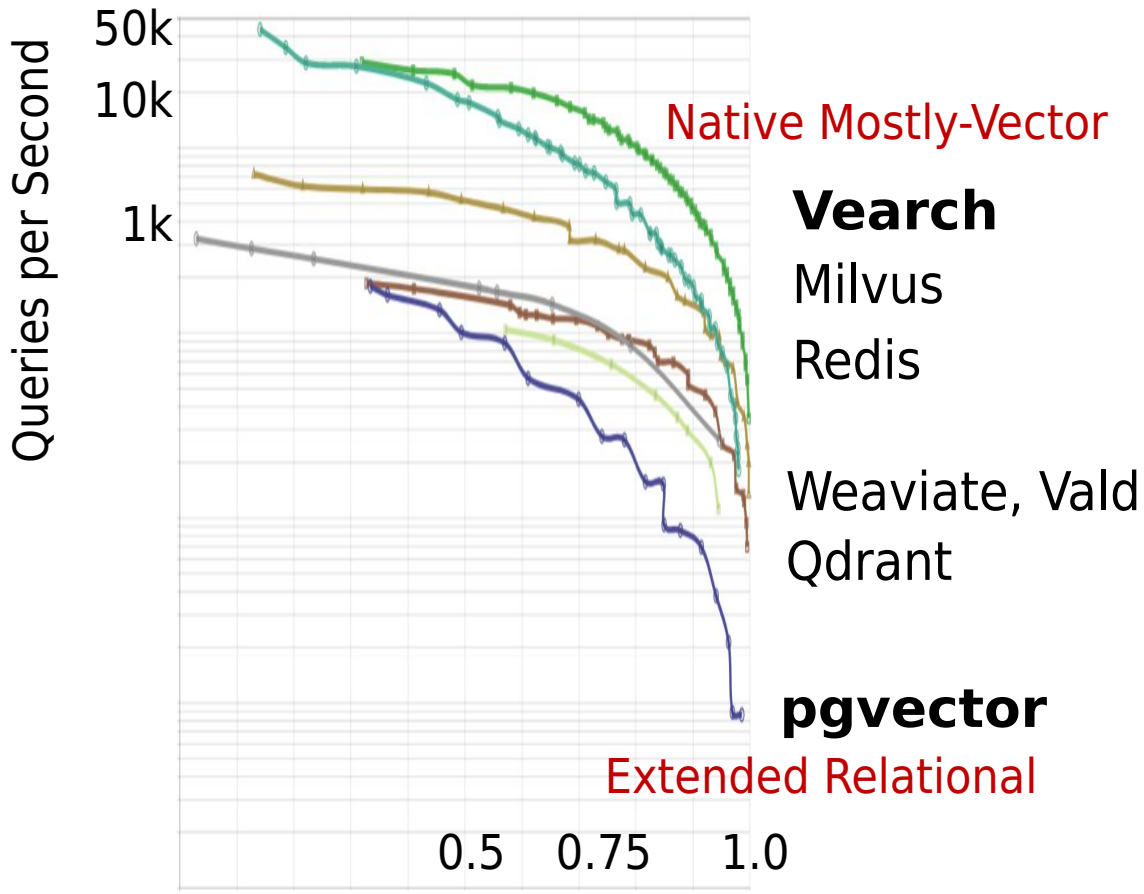
Extended Systems

Integrate vector search capabilities within existing data management system

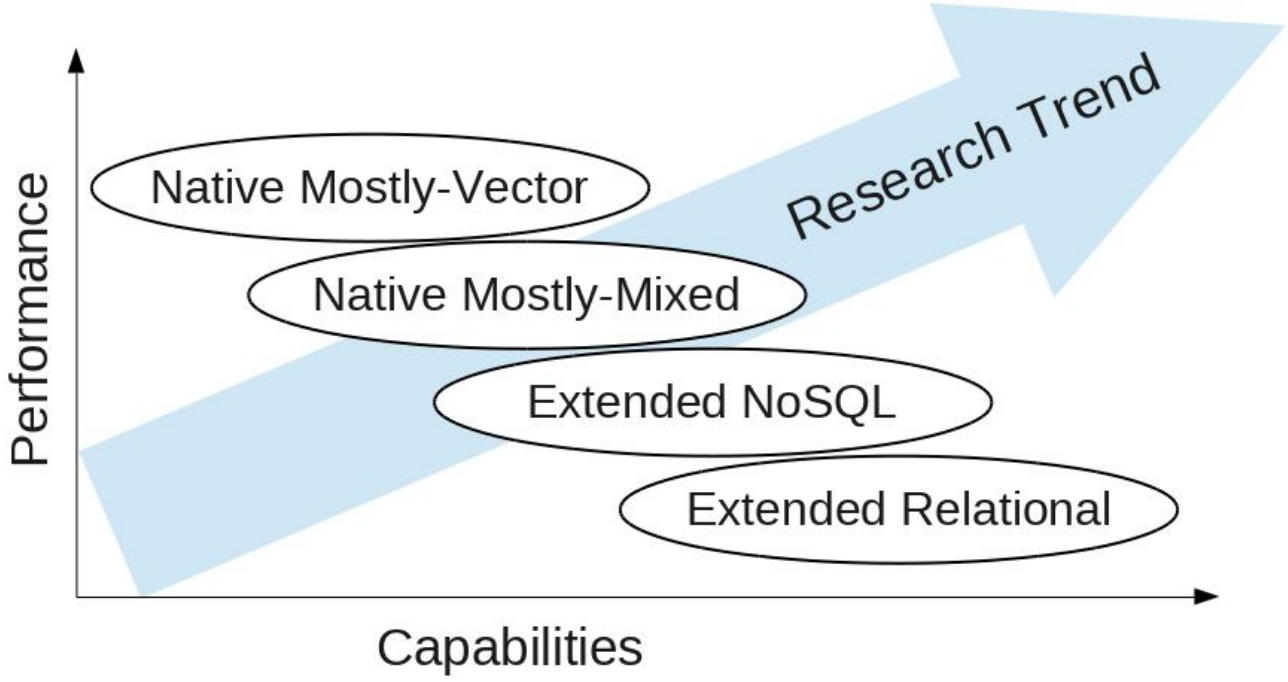
	Query Processor	Storage & Indexing	Query Optimizer	Query Executor
Redis	• k-NN • Range NN • Predicated k-NN • *Key-value queries*	• Key-Value Store • HNSW, Vamana • Quantization	N/A	N/A
PostgreSQL (PASE, pgvector)	• k-NN • Range NN • Predicated k-NN • Full Text Search • *Relational queries*	• Page-Based Storage • Shards & Replicas • HNSW, IVF • Quantization	• Cost-based predicated query planning	• MVCC • Scatter-gather • Log-backed durable writes • etc.
GaussDB	• k-NN • Range NN • Predicated k-NN • *Relational queries*	• Page-Based Storage • Shards & Replicas • HNSW, IVF, Hybrid Index • Quantization	• Cost-based predicated query planning	• GaussDB Transaction Manager • Scatter-gather • Log-backed durable writes • Cache & SIMD acceleration • NPU/GPU acceleration

Summary & Looking Forward

Existing systems are designed to provide high-performance search with various system-level features



(a) Comparison of some existing systems
[ann-benchmarks.com, 2024]

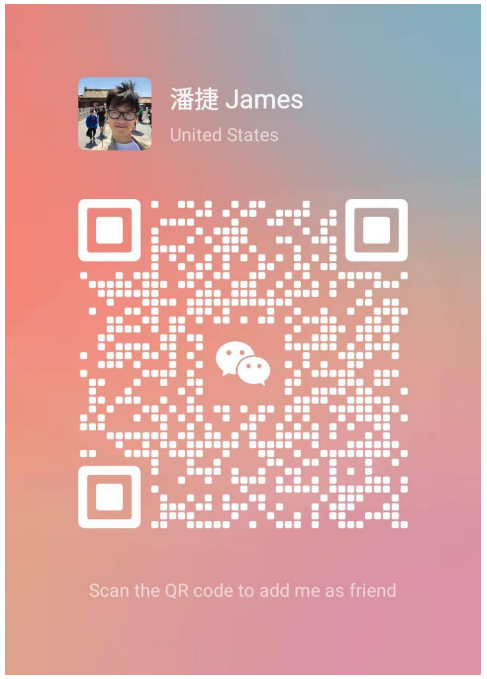


(b) Future research trend

Thanks!

Survey of Vector Database Management Systems
VLDB Journal (2024)

Survey of Vector Database Management Systems
arXiv:2310.14021



James Pan
jpan@tsinghua.edu.cn

Slides:

https://dbgroup.cs.tsinghua.edu.cn/ligl/activities.html